Databases – Lecture 1

Yonghao Lee

March 23, 2026

Contents

1 Data Modeling **2**
1.1 The Database Design Process . . . 2
1.2 Entity–Relationship (ER) Diagrams . . . 2
1.2.1 Entities and Entity Sets . . . 2
1.2.2 Attributes . . . 3
1.2.3 Relationships and Relationship Sets . . . 3
1.2.4 Recursive Relationship Sets . . . 4
1.2.5 Attributes of Relationship Sets . . . 4
1.2.6 n-ary Relationship Sets . . . 6
1.3 Multiplicities in Relationships . . . 7
1.3.1 Many-to-Many Relationships . . . 7
1.3.2 Many-to-One and One-to-Many Relationships . . . 7
1.3.3 One-to-One Relationships . . . 8
1.3.4 Example: Where do we put the arrow for Students and Universities? 8
1.3.5 Example: Where do we put the arrow for Father and Child? . . . 9
1.3.6 Multiplicities in Non-binary Relationships . . . 9
1.3.7 Multiple Arrows in Generalized Relationships . . . 10
1.3.8 Example: Multiplicity in a Ternary Relationship . . . 11

1 Data Modeling

1.1 The Database Design Process

Designing a database involves four main steps:

1. **Requirement Analysis.** Determine what data is stored, how it is used, and what constraints apply. For example, in a university setting we might store information about students, lecturers, courses, and enrolments; constraints include the uniqueness of student IDs and limits on class sizes.

2. **Conceptual Design.** Produce a high-level, implementation-independent description of the data. The standard tools are *Entity–Relationship (ER) diagrams* and UML class diagrams.

3. **Logical Design.** Translate the conceptual model into a concrete schema—typically a set of *relational tables* with column definitions, keys, and integrity constraints.

4. **Physical Design.** Decide how the logical schema maps onto storage: file organisation, partitioning, index types, buffer management, etc.

1.2 Entity–Relationship (ER) Diagrams

An ER diagram is a graphical notation for modelling data at the conceptual level. It captures three main concepts: **entities**, **attributes**, and **relationships**.

1.2.1 Entities and Entity Sets

Definition 1.1 Entity

An **entity** is a real-world object that is distinguishable from all other objects. It possesses a set of properties (attributes) whose values uniquely identify it.

Examples:
- A specific student (identified by student ID).
- A specific course offering (identified by course number + semester).

Non-example: Two identical, unmarked leaves on a tree—without distinguishing properties we cannot tell them apart, so they do not form separate entities in our model.

Definition 1.2 Entity Set

An **entity set** is a set of entities of the same type, sharing the same attribute *structure* (i.e. the same attribute names, but not necessarily the same values). It is represented in the ER diagram as a **rectangle**.

Example: Students is the entity set; each individual student is an entity in that set.

1.2.2 Attributes

Definition 1.3 Attribute

An **attribute** is a property that describes an entity or a relationship instance. Every member of a given entity set (or relationship set) has the same set of attributes. In an ER diagram an attribute is drawn as an **oval** connected to its entity set or relationship set.

Remark

Every entity in the set must have a non-null value for all attributes.
For example, consider an Actor entity set with attributes `id`, `name`, `birthday`, and `address`. We should *not* include an attribute `spouse`, since some actors do not have spouses (the value would be null). Likewise, all attributes must be single-valued: if an actor can have multiple email addresses, we cannot declare `email` as a simple attribute of the set.

Definition 1.4 Key

A **key** is a minimal set of attributes that uniquely identifies an entity in the set. Every entity set must have a key. In the ER diagram the key attributes are <u>underlined</u>.

Example: In the Actor entity set, `id` can serve as the key, since it alone is sufficient to uniquely identify each actor.

Remark

Whether `name` and `address` together can serve as a key depends on the context. If we know that two actors may share both the same name and the same address, then {`name, address`} cannot be a key.

Remark

Can {`id, name`} be a key? No—because `id` alone already uniquely identifies each entity, so {`id, name`} is not minimal.

1.2.3 Relationships and Relationship Sets

Definition 1.5 Relationship

A **relationship** is an association among two or more entities. A **relationship set** is the collection of all relationships of the same type; it is drawn as a **diamond** in an ER diagram.

Formally, a relationship set is a subset of a Cartesian product of entity sets. For example:

$$\mathsf{StarsIn} \subseteq \mathsf{Actor} \times \mathsf{Film}$$

where $\mathsf{Actor} = \{a_1, a_2, \ldots\}$ and $\mathsf{Film} = \{f_1, f_2, \ldots\}$ are entity sets, and each element of StarsIn is a pair (a_i, f_j) recording that actor a_i stars in film f_j.

Example: The relationship set Enrols associates entities from Students with entities from Courses—for instance, the entity Yosef (a student) enrols in the entity DB (a course).

1.2.4 Recursive Relationship Sets

Sometimes the same entity set participates in a relationship set *more than once*. This is called a **recursive** (or **self-referencing**) relationship.

Definition 1.6 Recursive Relationship

A **recursive relationship set** is a relationship set in which the same entity set appears in multiple roles. Because the entity set is used more than once, we must label the connecting lines with **role names** to distinguish the different participations.

Example: Consider the entity set Employee with a relationship set Manages. Each instance of Manages pairs two employees—one in the role of *manager* and one in the role of *worker*. Formally:

$$\mathsf{Manages} \subseteq \underbrace{\mathsf{Employee}}_{\text{manager}} \times \underbrace{\mathsf{Employee}}_{\text{worker}}$$

In the ER diagram, two lines connect Employee to the Manages diamond, and each line is annotated with its role (*manager* or *worker*).

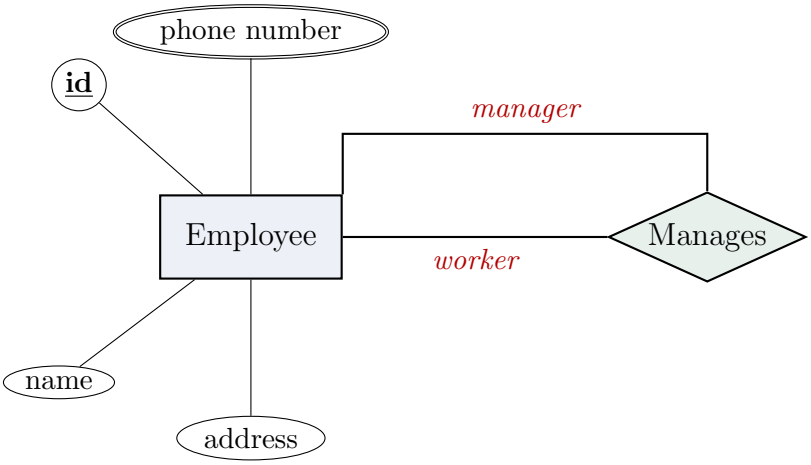

Figure 1: Recursive relationship: Employee Manages Employee, with role labels *manager* and *worker* on the connecting lines. Note that `phone number` is drawn with a double oval since it is multivalued.

1.2.5 Attributes of Relationship Sets

Relationship sets can have their own attributes, but the placement of an attribute determines its meaning. The following example illustrates why this matters.

Where does `salary` belong?

Suppose we want to record an actor's salary. There are three possible placements, each with a different semantic meaning:

1. **On the Actor entity set.** Each actor has a single, fixed salary (e.g. a monthly paycheck), independent of any film.
2. **On the Film entity set.** Each film has one salary value, meaning every actor in that film earns the same amount—clearly unrealistic.

3. **On the ActedIn relationship set** (ActedIn $\subseteq$ Actor $\times$ Film). Each *(actor, film) pair* has its own salary. An actor can earn a different amount in each film. This is usually the correct choice.

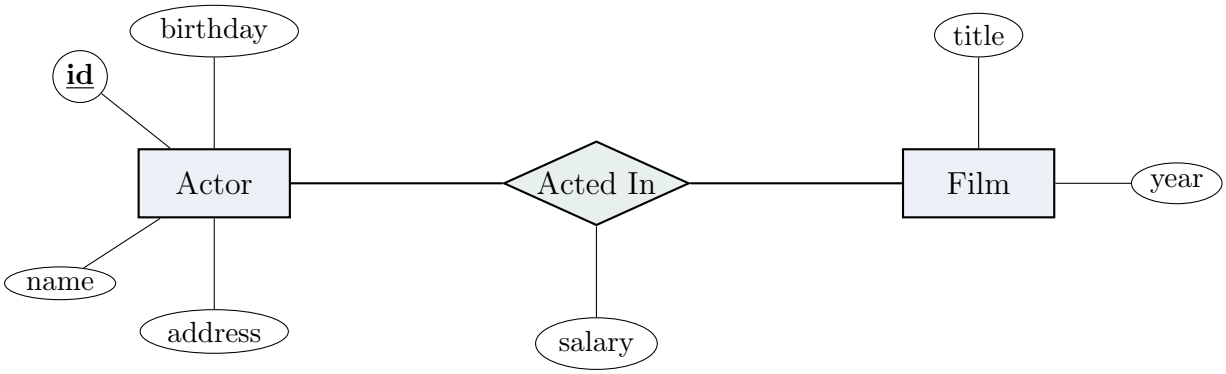

Figure 2: The `salary` attribute attached to the ActedIn relationship set: each (actor, film) pair has its own salary.

Constraint on relationship attributes

Just like entity attributes, attributes of a relationship set must be **single-valued** and **non-null** for every relationship instance. That is, for each pair $(a_i, f_j) \in$ ActedIn, the attribute `salary` must have exactly one value.

Now consider adding a `role` attribute (e.g. "villain", "hero") to the ActedIn relationship.

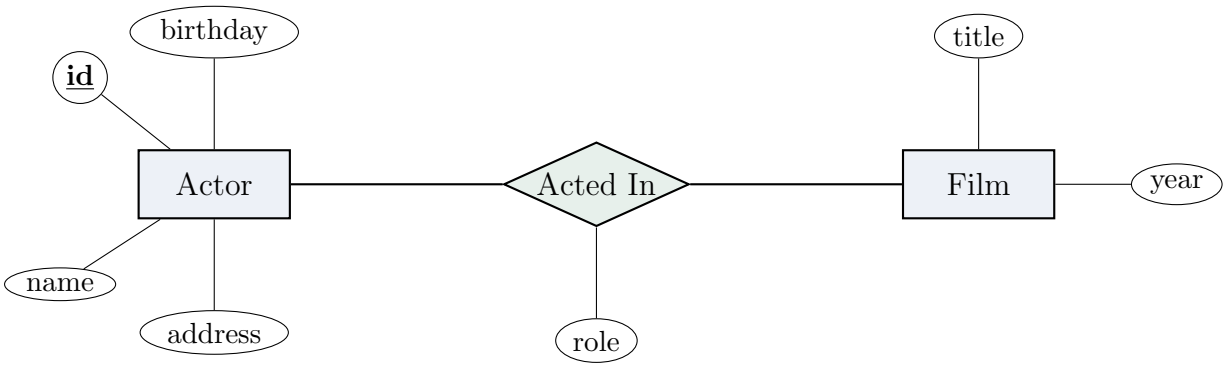

Figure 3: Attempting to place `role` as an attribute of ActedIn.

Why `role` fails as a relationship attribute

Suppose an actor plays *two roles* in the same film (e.g. a double role). The relationship instance is the pair (a_i, f_j), but we would need to store two values for `role`—violating the single-valued requirement. Similarly, if an actor appears in a film without a named role, the value would be null—violating the non-null requirement.

Therefore `role` **cannot** be modelled as a simple attribute of ActedIn.

1.2.6 n-ary Relationship Sets

So far, every relationship set we have seen has been **binary**—it connects exactly two entity sets. In general, a relationship set can connect *any number* of entity sets.

Definition 1.7 n-ary Relationship Set

An **n-ary relationship set** R is a relationship among n entity sets $E_1, E_2, \ldots, E_n$. Formally:

$$R \subseteq E_1 \times E_2 \times \cdots \times E_n$$

Each member of R is an n-tuple $(e_1, e_2, \ldots, e_n)$. The most common cases are:

- $n = 2$: **binary** — members are pairs.
- $n = 3$: **ternary** — members are triples.

Why not just use multiple binary relationships?

Consider the fact "Actor A was directed by Director D in Film F." This involves three entities *simultaneously*. If we used two separate binary relationships (ActedIn and DirectedFilm), we could record that A acted in F and that D directed F, but we would lose the information about *which director directed which actor in which specific film*. A ternary relationship captures the full three-way association in one tuple.

Example. Consider three entity sets: Actor, Director, and Film. The ternary relationship set

$$\mathsf{DirectedActorIn} \subseteq \mathsf{Actor} \times \mathsf{Director} \times \mathsf{Film}$$

records triples (a_i, d_j, f_k) meaning "actor a_i was directed by director d_j in film f_k."

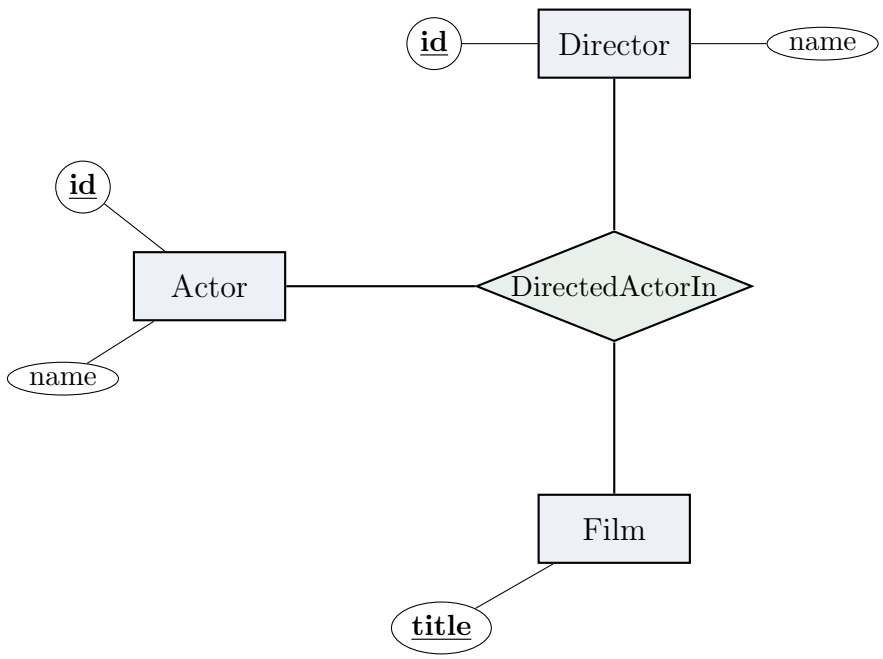

Figure 4: Ternary relationship: DirectedActorIn $\subseteq$ Actor $\times$ Director $\times$ Film.

Counting tuples

Suppose $|\mathsf{Actor}| = 2$, $|\mathsf{Director}| = 3$, and $|\mathsf{Film}| = 4$. The full Cartesian product contains $2 \times 3 \times 4 = 24$ possible triples, so DirectedActorIn can have **at most 24** members.

Note that each member is a *triple*, not a pair—there are no pairs in a ternary relationship.

1.3 Multiplicities in Relationships

A crucial aspect of database conceptual design is defining the **multiplicity** (or **cardinality ratio**) of a relationship set. Multiplicity specifies the number of relationship instances that a specific entity can participate in.

1.3.1 Many-to-Many Relationships

The most general form of a binary relationship is **many-to-many** (often denoted as $M : N$ or $N : M$). In a many-to-many relationship between entity sets E and F, a single entity in E can be associated with any number of entities in F, and vice versa.

Consider the following standard representation:

Figure 5: A basic many-to-many relationship set R between entity sets E and F.

Number of possible pairs

If entity set E contains n entities ($|E| = n$) and entity set F contains m entities ($|F| = m$), the number of possible pairs in R falls in the range $[0, n \times m]$.

- **Maximum ($n \times m$):** This occurs if every single entity in E is related to every single entity in F (the full Cartesian product).
- **Minimum (0):** Because R is simply a subset of $E \times F$ ($R \subseteq E \times F$), the empty set is a mathematically valid subset. In practice, this just means that at a given time, no entities currently participate in the relationship.

1.3.2 Many-to-One and One-to-Many Relationships

Sometimes we want to emphasize a stricter constraint: that an entity can participate *at most once* in a relationship set. In standard ER notation, we indicate this by drawing an **arrow** pointing towards the entity set where the "one" constraint applies.

If we draw an arrow from the relationship set R towards F, it indicates that a member of E may participate in R **at most once**. Consequently, each entity in E is associated with at most one entity in F.

However, a member of F can still participate any number of times, meaning a single entity in F can be associated with many entities in E. This is called a **many-to-one** relationship from E to F (or equivalently, a **one-to-many** relationship from F to E).

Figure 6: A many-to-one relationship from E to F. The arrow indicates that an entity in E relates to at most one entity in F.

Number of possible pairs

If entity set E contains n entities ($|E| = n$) and the arrow points to F, the number of possible pairs in R falls in the range $[0, n]$.

- **Maximum (n):** This occurs if every single entity in E participates exactly once. Since each E can only map to one F, there can be at most n tuples in the relationship set.
- **Minimum (0):** As with many-to-many relationships, the set might be empty if no entities in E are currently participating in R.

1.3.3 One-to-One Relationships

If we place an arrow pointing towards *both* entity sets, it means that a member of E may participate in R at most once, **and** a member of F may also participate in R at most once. This creates a **one-to-one** (1:1) relationship.

Figure 7: A one-to-one relationship. The arrows indicate that an entity in E maps to at most one entity in F, and an entity in F maps to at most one entity in E.

Number of possible pairs

If entity set E contains n entities and entity set F contains m entities, the number of possible pairs in R falls in the range $[0, \min(n, m)]$.
Because no entity in either set can be used more than once, the maximum number of pairs is strictly limited by the smaller of the two entity sets.

1.3.4 Example: Where do we put the arrow for Students and Universities?

Suppose we have an entity set Student, an entity set University, and a relationship set StudiesAt.

In a typical conceptual model, a university enrolls many students, but a student pursues their primary degree at **at most one** university. Therefore, for any given student, there is at most one associated university. We place the arrow pointing towards University.

Figure 8: Many-to-one relationship: A student studies at at most one university.

1.3.5 Example: Where do we put the arrow for Father and Child?

Consider the entity set Person and a recursive relationship set FatherOf. The two lines connecting them are labelled with the roles *father* and *child*.

A biological father can have many children, but a child has **exactly one** biological father. Because a child maps to at most one father, the constraint applies to the father role. Therefore, the arrow must point from the relationship set to the Person entity set along the line labelled **father**.

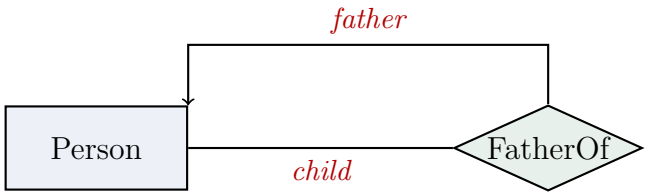

Figure 9: Recursive relationship FatherOf. The arrow on the *father* line indicates that a given person in the *child* role has at most one person in the *father* role.

1.3.6 Multiplicities in Non-binary Relationships

Suppose we have a relationship set R connecting n entity sets $E_1, E_2, \ldots, E_n$ along with an entity set F. We write the Cartesian product subset as:

$$R \subseteq E_1 \times E_2 \times \cdots \times E_n \times F$$

If we draw an arrow from the relationship diamond R to the entity set F, it indicates that a specific combination of entities from E_1 through E_n is associated with **at most one** entity in F.

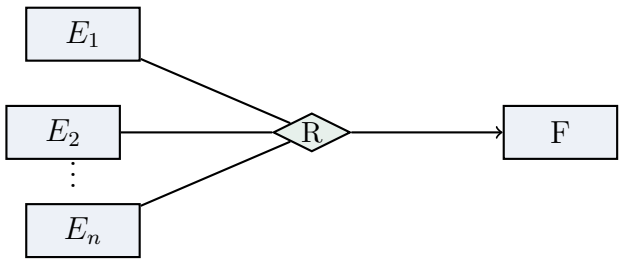

Figure 10: A generalized $(n+1)$-ary relationship where $E_1, \ldots, E_n$ jointly determine at most one F.

Definition 1.8 Formal Multiplicity Constraint

For an $(n+1)$-ary relationship set R with an arrow pointing to F, the "at most one" constraint is defined mathematically as follows:

If we have two tuples in R:

$$(e_1, \ldots, e_n, f) \in R \quad \text{and} \quad (e_1, \ldots, e_n, f') \in R$$

then it must be true that:

$$f = f'$$

Connection to Functional Dependencies

In the context of relational database theory, this mathematical property is known as a **functional dependency**. The set of participating entities $\{e_1, \ldots, e_n\}$ acts as a composite key that functionally determines f.

1.3.7 Multiple Arrows in Generalized Relationships

We can extend the formal definition to relationships that have *multiple* arrows pointing to different entity sets. Suppose a relationship set R connects n entity sets without arrows $(E_1, \ldots, E_n)$ and m entity sets with arrows $(F_1, \ldots, F_m)$.

The rule is that the "at most one" constraint applies to *each* arrow individually. For any specific target set F_i, fixing a specific combination of entities from *all other* participating sets uniquely determines at most one entity in F_i.

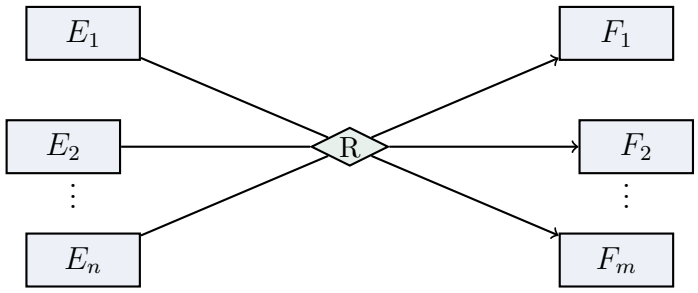

Figure 11: An $(n+m)$-ary relationship with arrows pointing to $F_1, \ldots, F_m$. The constraint applies to each F_i individually.

Definition 1.9 Formal Multiplicity for Multiple Arrows

For an $(n+m)$-ary relationship set R where arrows point towards $F_1, \ldots, F_m$, the following mathematical constraint must hold for **all** $1 \leq i \leq m$:

If we have two tuples in R that share the exact same participating entities in every position *except* potentially the F_i position:

$$(e_1, \ldots, e_n, f_1, \ldots, f_{i-1}, \mathbf{f_i}, f_{i+1}, \ldots, f_m) \in R$$

and

$$(e_1, \ldots, e_n, f_1, \ldots, f_{i-1}, \mathbf{f'_i}, f_{i+1}, \ldots, f_m) \in R$$

then it must be true that the entities in the F_i position are identical:

$$f_i = f'_i$$

Maximum Tuples with Multiple Arrows

Consider a relationship R connecting entity sets A, B, C, and D, where $|A| = a$, $|B| = b$, $|C| = c$, and $|D| = d$. Suppose there are arrows pointing to C and D.
This means the combination of (A, B, D) uniquely determines C, **and** the combination of (A, B, C) uniquely determines D. Because both constraints must hold, the maximum number of possible relationship pairs in R is constrained by whichever target set is smaller.

> Therefore, the number of possible pairs in R falls in the range $[0, a \cdot b \cdot \min(c, d)]$.

1.3.8 Example: Multiplicity in a Ternary Relationship

Let's apply our formal multiplicity rules to a concrete ternary relationship. Consider the relationship set Studies connecting Student, Teacher, and Course.

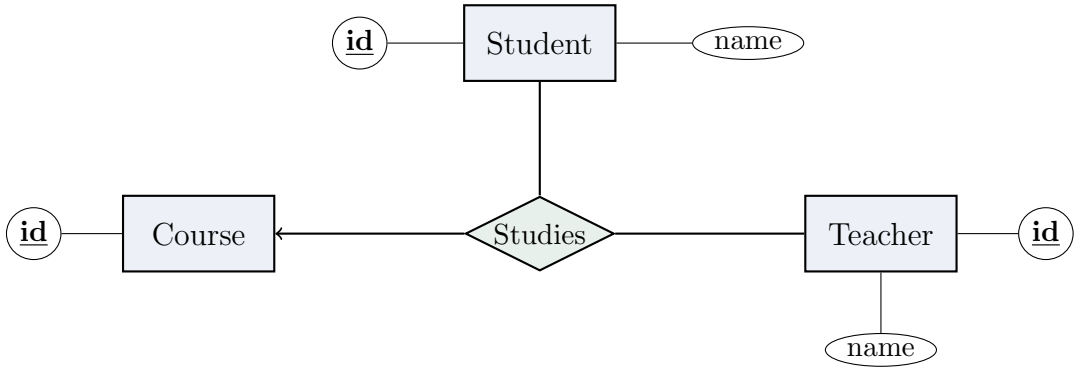

Figure 12: The ternary relationship Studies with an arrow pointing to Course.

Scenario 1: Arrow pointing to Course (As drawn) According to our rule, the combination of the *other* entities (Student and Teacher) determines at most one Course.

- **Mathematical meaning:**
  If $(\text{Student}_A, \text{Teacher}_B, \text{Course}_X) \in$ Studies and $(\text{Student}_A, \text{Teacher}_B, \text{Course}_Y) \in$ Studies, then $\text{Course}_X = \text{Course}_Y$.
- **Real-world meaning:** If a specific student has a specific teacher, it is for exactly one course. A student cannot take multiple different courses with the same teacher. In other words, a given student-teacher pair is associated with at most one course.

Scenario 2: Arrows pointing to both Student and Course What if the relationship has multiple multiplicity constraints? Let's modify the diagram so that an arrow points to Student *and* an arrow points to Course.

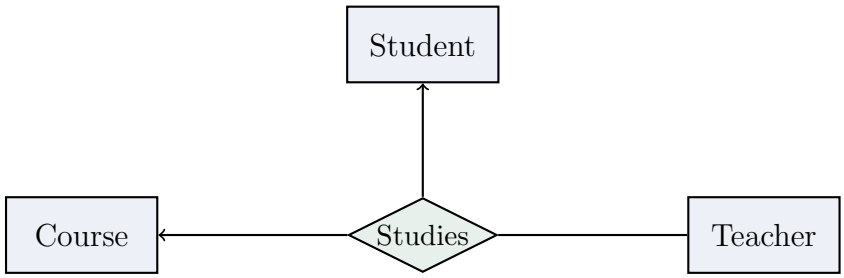

Figure 13: The ternary relationship Studies with two arrows, pointing to both Course and Student.

Here, we must interpret the two arrows entirely separately, applying the "at most one" rule to each target entity set:

- **Interpreting the arrow to Course:**
  The combination of Student and Teacher determines at most one Course.

- *Mathematical:* If $(s, t, c_1) \in \mathsf{Studies}$ and $(s, t, c_2) \in \mathsf{Studies}$, then $c_1 = c_2$.
- *Real-world:* As established, a specific student is taught by a specific teacher in at most one course.

- **Interpreting the arrow to Student:**
  The combination of Teacher and Course determines at most one Student.
  - *Mathematical:* If $(s_1, t, c) \in \mathsf{Studies}$ and $(s_2, t, c) \in \mathsf{Studies}$, then $s_1 = s_2$.
  - *Real-world:* A specific teacher teaching a specific course has at most one student enrolled. This represents a private tutoring arrangement where each course offering is strictly one-on-one.

Combining the constraints

When we combine both rules, the model tells us: A teacher gives a private, one-on-one class to a single student (arrow to Student), and that student never takes another class with that same teacher (arrow to Course).